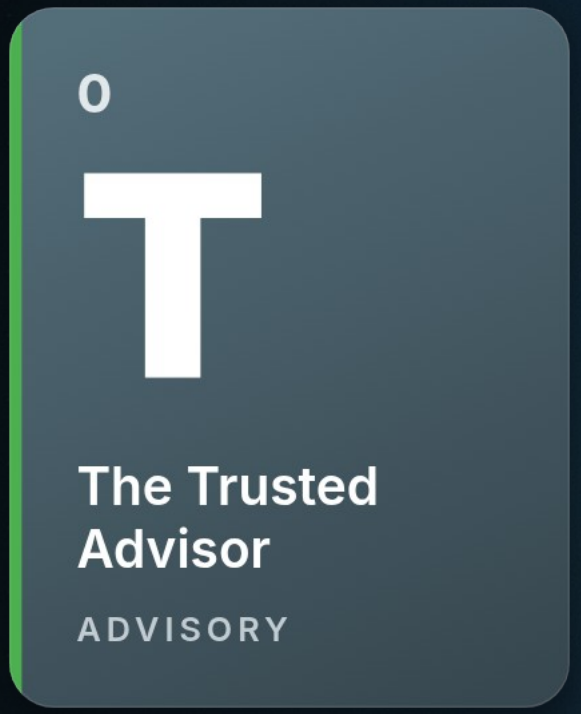


TICINO DATA CONFERENCE 2026 · BIOGGIO

Boss Your Agents: How AI Secures What AI Teams Build

Matthias Falland

MICROSOFT DATA PLATFORM MVP · THE TRUSTED ADVISOR

ABOUT ME

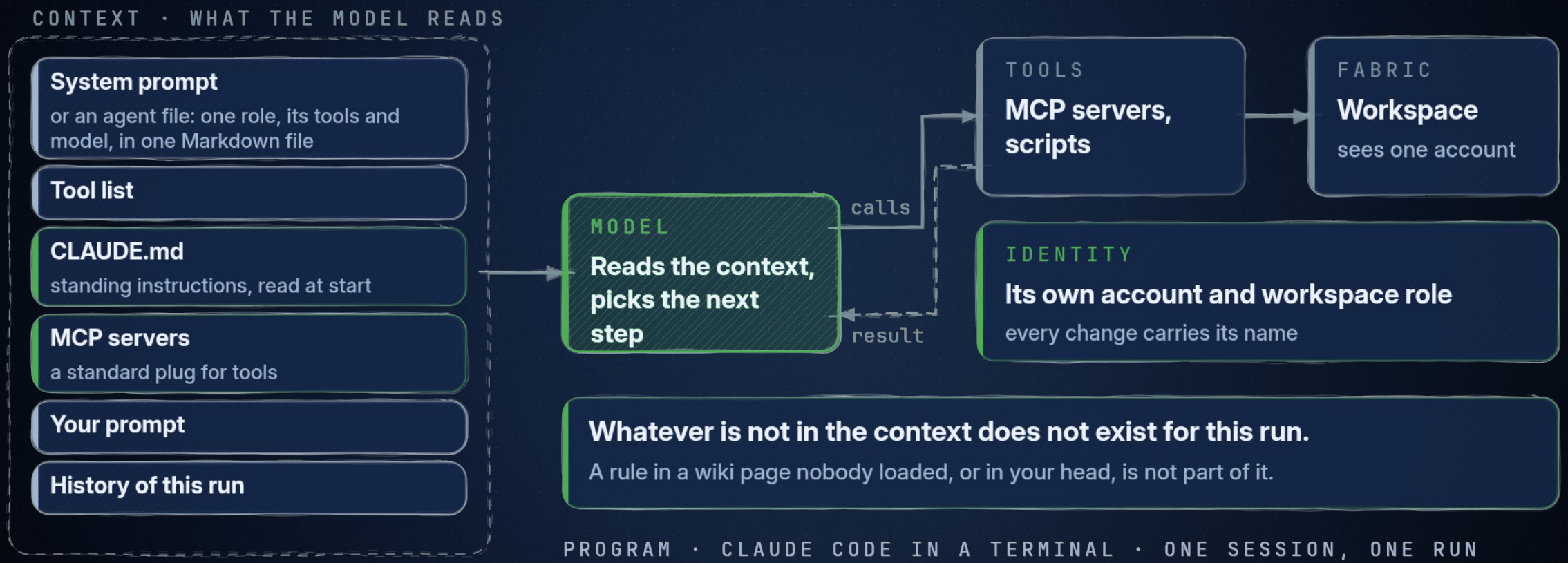
Matthias Falland

The Trusted Advisor · Microsoft Data Platform MVP since 2018

- **Microsoft MVP** Data Platform, since 2018
- **Fabric Friday** weekly on YouTube, one Fabric element per episode
- **Fabric Architecture Podcast** weekly, with an AI co-host
- **Fabric Periodic Table** fabricperiodictable.com, every Fabric item
- **Meetups** Zurich, Hamburg, Basel: over 5,000 members

LinkedIn: matthias-falland
Repo of this talk →

What an agent session is



00

Why rules go missing

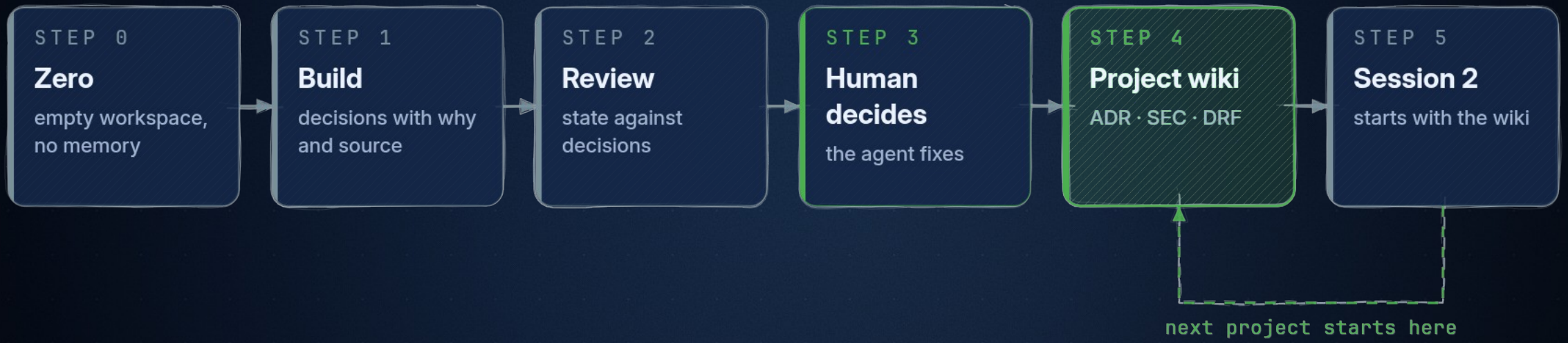
context, not intelligence

- The model follows what it reads: your rules are in a wiki and in people's heads
- Without them: missing row-level security, inconsistent names, undocumented decisions
- A setup and five steps put the project's knowledge into the context, and check it

THE ARC

Zero to hero: a setup and five steps

THE MAIN CHARACTER IS THE PROJECT'S KNOWLEDGE



ADR = ARCHITECTURE DECISION RECORD · SEC = SECURITY FINDING · DRF = DRIFT BASELINE

NOTHING SEEDED · EVERY STEP IS A FILE OR A COMMAND YOU CAN REDO

Zero

What does a session need before it builds anything?

37

WS

Workspace

GOVERN

STEP 0 · MECHANISM

Three layers, set before the first prompt

	IDENTITY	WORKSPACE ROLE	TOOLS AND MCP SCOPE
Builder session	**Its own account** never yours, never an admin	**Contributor, one workspace** not in the tenant's "Create workspaces" group	**Write tools, one workspace** plus Microsoft Learn and a wiki
Reviewers	**Their own account** separate from the builder	**Contributor, to read definitions** a hook checks its commands	**Reads definitions + wiki** the session blocks other writes
Human	**You** the only one who decides	**Approves and signs** the baseline	**Reads the report** answers open questions

FACT CARD · STEP 0

Zero

WHAT

Before the first prompt, each agent gets its own identity, the smallest role, Microsoft Learn and a wiki (a folder in the repo is enough).

GOOD WHEN

You start a project, or hand an existing one to an agent for the first time.

IN YOUR PROJECT

```
Identity per agent
  never yours, never an admin
MCP server: Microsoft Learn
A wiki: a folder in the repo
CLAUDE.md, one instruction:
  record every decision: what,
  why, alternatives, source
Deny the builder access
  to the reviewers' files
```

TAKEAWAY

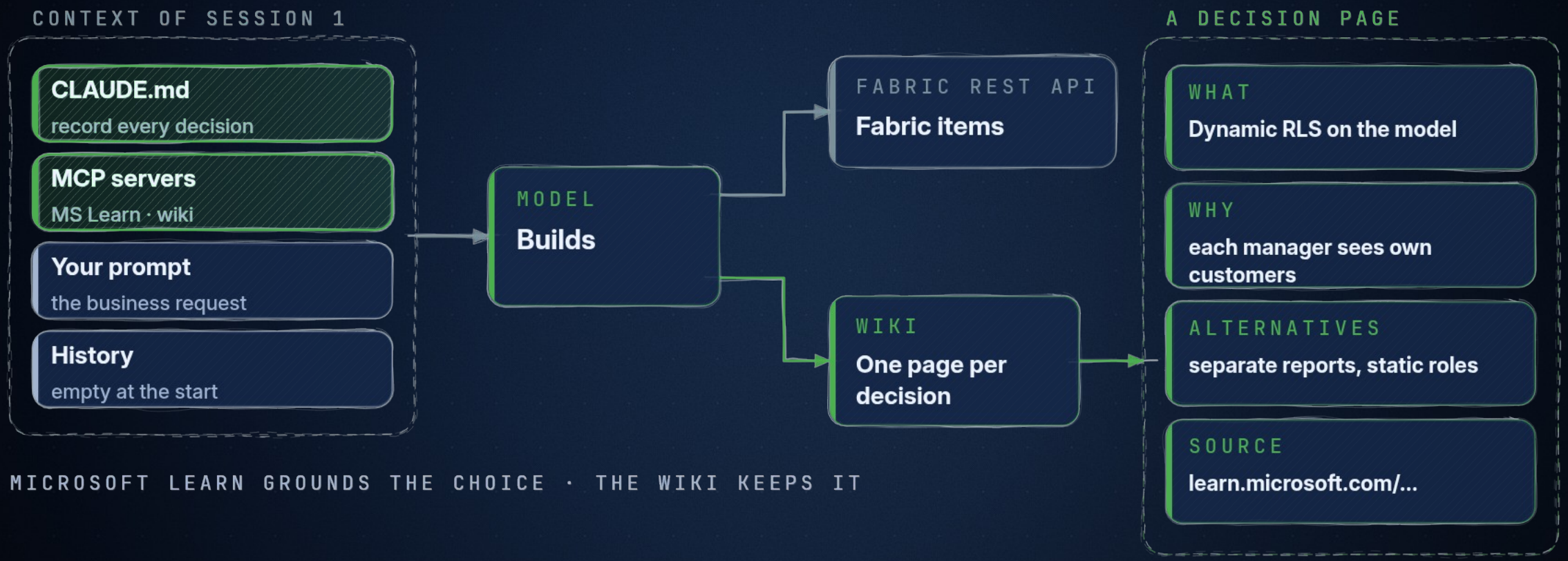
A decision log needs an instruction and a place to write.

1 Build

How does an agent leave its decisions behind?

STEP 1 · MECHANISM

Every decision becomes a page



MICROSOFT LEARN GROUNDS THE CHOICE · THE WIKI KEEPS IT

DEMO · HOW IT RUNS

One real run, recorded in one go

- Microsoft Fabric, an F2 capacity, two empty workspaces
- Claude Code; a small toolkit calls the Fabric REST API with each agent's own account
- Waiting time is cut; the right column says what to look for
- From step 3 on, live from the state this run left

Step 1 · Session 1 builds and records its decisions

Direct Lake on OneLake (not on SQL endpoint)

Decision: The semantic model `Sales` uses Direct Lake on OneLake: shared expression `AzureStorage.DataLake("https://onelake.dfs.fabric.microsoft.com/<ws id>/<lakehouse id>")`, partitions `mode: directLake`.

Why: Microsoft recommends Direct Lake on OneLake for new semantic models; it never falls back to DirectQuery, and semantic-model RLS is supported.

Alternatives: Direct Lake on SQL analytics endpoint (can silently fall back to DirectQuery); Import (requires data refresh copies, not what was asked).

Source: **https://learn.microsoft.com/fabric/fundamentals/direct-lake-how-it-works#directquery-fallback** https://learn.microsoft.com/fabric/fundamentals/direct-lake-how-it-works#directquery-fallback , **https://learn.microsoft.com/fabric/fundamentals/direct-lake-develop#power-query-connectors** https://learn.microsoft.com/fabric/fundamentals/direct-lake-develop#power-query-connectors

(END)

④ Takeaway

Read the pages yourself. A good page says what was decided, why, what else was possible, and where it is documented.

FACT CARD · STEP 1

Build with a decision log

WHAT

The builder works on the request and writes one wiki page per decision while it builds.

GOOD WHEN

An agent makes a choice that someone will ask about later.

IN YOUR PROJECT

```
The request as written
  no hidden rules
One wiki page per decision
  what · why ·
  alternatives · source
Written when decided,
  not at the end
```

TAKEAWAY

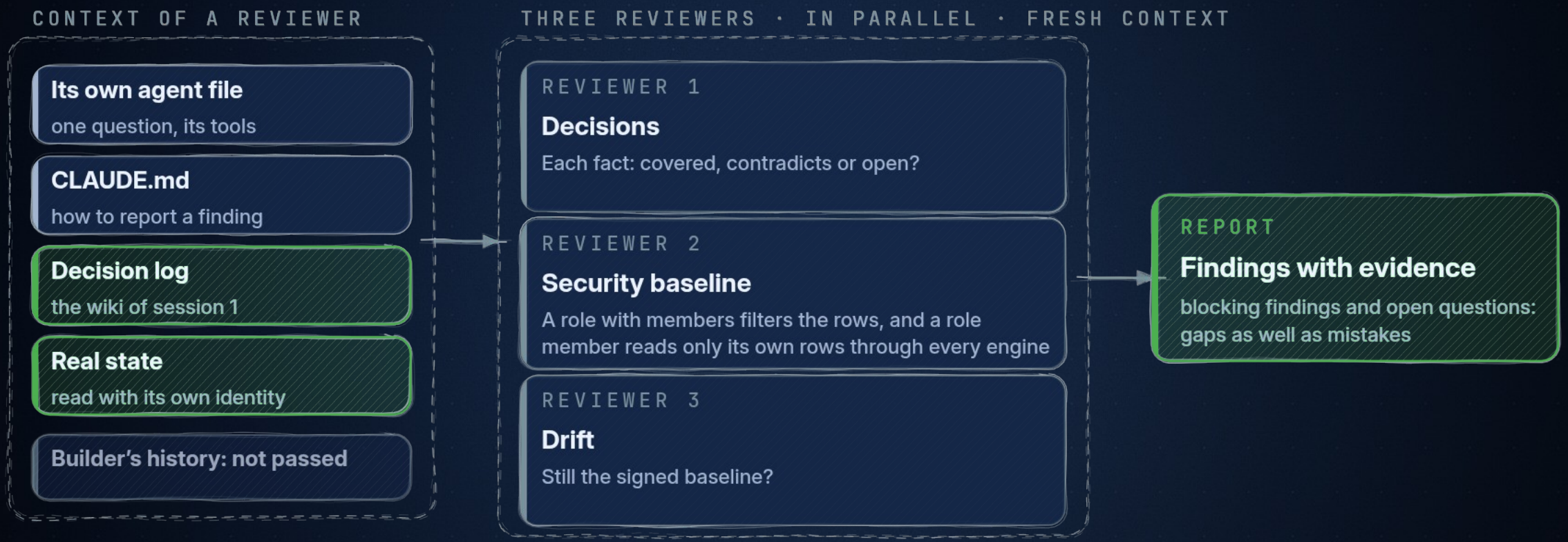
A good page says what, why, what else was possible, and the source.

2
Review

Does the state match the decisions, and does the security hold?

STEP 2 · MECHANISM

What the reviewer gets



STEP 2 · EVIDENCE

A finding is a claim, evidence and a remedy

CLAIM

What the reviewer says

one sentence, checkable

EVIDENCE

What was read, by whom

item, definition, query · which identity

REMEDIATION

What would close it

a change, or a decision

SEC-020 · BLOCKING · FROM THE REVIEW

CLAIM

RLS bypass: a role member reads 60 of 60 rows through the SQL endpoint

EVIDENCE

A test identity in the RLS role read the customers table through the SQL endpoint

60 rows, 40 of them belong to other account managers

REMEDIATION

Apply the same rule wherever the table can be read

a security policy in the SQL endpoint, or OneLake security

MODEL RLS APPLIES ONLY INSIDE THE MODEL

Step 2 · Three reviewers read the real state

```
$ sed -n '/^## Open/,/^## Info/p' reports/latest-gate.md | glow -w 84 -

  ## Open questions (a human decides)

  ### Q-1 · open · how items are named

  What naming convention should items follow (why sales_lh and Sales, no prefix/suffix
  or environment tag)?

  • Evidence: F1: items built: Lakehouse «sales_lh»; SemanticModel «Sales». Names
  appear only on pages about other choices. No decision page covers it.
  • Remediation: a human decides (one sentence) and the decision goes into the log
  • Reviewer: decisions

  ## Info

$
```

④ Takeaway

Test what a rule does, not only that it exists. A choice nobody made, like item names, becomes a question for a person.

FACT CARD · STEP 2

Review with a fresh context

WHAT

Reviewers with their own identity and a fresh context read the deployed state and the decision log, and test what a role member really sees.

GOOD WHEN

Before anyone builds on a result, and after every change.

IN YOUR PROJECT

```
One agent file per reviewer
  one question each
Its own identity
  reads the real state
A test identity in the role
  reads through every engine
A finding:
  claim · evidence · remedy
No decision? an open question
```

TAKEAWAY

Test the effect: model RLS applies only inside the model.

DEMO · LIVE FROM HERE

The same run, continued live

- Workspace one is as the review found it, nothing rebuilt
- I decide; the agent fixes; the review runs again; I sign
- An agent writes the wiki; session two starts with it
- If the network fails, the recording of the same run takes over

3

Human decides

Who decides, who fixes, and who proves it?

STEP 3 · MECHANISM

You decide, the agent fixes, the review proves

THE HUMAN DECIDES, IN OWN WORDS

Q-1 · OPEN QUESTION

Items use type prefixes: lh_, sm_, rpt_, nb_

so anyone can tell an item's type from its name

SEC-020 · BLOCKING FINDING

The account-manager rule applies wherever the table can be read: in the model and in the SQL endpoint

the scope of the rule, not the code

YOU DECIDE WHAT BECOMES A RULE

THE AGENT FIXES

Only what you decided

the same rule in the SQL endpoint, as a security policy

THE REVIEW PROVES

The probe reads 20 of 60 rows

none of another user · PASSED

THE HUMAN SIGNS

The reviewed definitions

with a key the agents can't reach · later changes show as drift

Step 3 · The person decides, the agent remediates, the review proves it

```
$ grep -o 'reads [0-9]* of [0-9]* rows[^|]*' reports/latest-gate.md | fold -s -w 84
reads 20 of 60 rows of [dbo].[customers] in lakehouse sales_lh through its SQL
analytics endpoint (u62solgdfuke...), none of another user, its own rows: 20
$ ./py gate/snapshot.py sign
snapshot signed by Matthias Falland: 2 items (Lakehouse/sales_lh, SemanticModel/Sales),
 8 definition parts -> /home/demo/.local/state/zero-to-hero/·····················
·············/snapshot.json, reports/baseline.json
$
```

④ **Takeaway**

A person decides, an agent remediates, the review measures. The person signs the state in the shell, not the agent.

FACT CARD · STEP 3

The human decides

WHAT

You decide each open question and blocking finding in one sentence; the agent fixes only that, the review runs again, you sign.

GOOD WHEN

A finding blocks, or a choice should become a rule.

IN YOUR PROJECT

```
One sentence per question
  or blocking finding
The agent fixes what
  you decided, nothing else
The review runs again:
  the fix is measured
Then sign the baseline
  key the agents can't reach
```

TAKEAWAY

You decide, the agent fixes, the review proves.

4 Project wiki

Where does the project's knowledge live?

STEP 4 · MECHANISM

One folder: index, three kinds of pages, rules

Decision log
session 1, with sources

Your decisions
two sentences

Review reports
SEC-020: 60 rows, then 20

Signed baseline
state at sign-off

WIKI/ · A FOLDER IN THE REPOSITORY

index.md
one line per page

ADR-006
account-manager rule in every engine

SEC-020
finding, measurement 60 → 20, decision

DRF-*
the signed baseline to compare with

rules/*.yaml
one per checkable ADR

RULES/ADR-006…YAML

```yaml
id: ADR-006
status: accepted
decided_by: the human
decision: "The account-manager
  rule applies wherever the
  table can be read: in the
  model and in the SQL
  endpoint. …"
check:
  rls_every_engine:
    table_with_columns:
      [account_manager_upn]
```

Step 4 · From log, report and decisions to the wiki

ADR-005 Item naming with type prefixes

Status

Accepted. Decided by the human: Matthias Falland, project owner, 2026-10-28 (tag `decided-by-human`). Applies from the next delivery.

Context

Open question Q-1 of the gate review of 2026-10-28 07:19 asked: "What naming convention should items follow (why sales_lh and Sales, no prefix/suffix or environment tag)?" The items had been built as Lakehouse «sales_lh» (type as suffix) and SemanticModel «Sales» (no type tag, capitalised). No convention had been decided.

Decision

"Items use type prefixes: lh_, sm_, rpt_, nb_, lower case, underscores, so anyone can tell an item's type from its name."

Why

So anyone can tell an item's type from its name. Consistent, predictable names help users and administrators.

Alternatives considered

Keep the names as deployed (`sales_lh`, `Sales`): type as suffix or missing, mixed case, no environment tag.

:

④ Takeaway

A rule belongs in the wiki when a reviewer can check it. Everything else stays a recorded decision.

FACT CARD · STEP 4

The project wiki

WHAT

Log, reviews and your decisions become one folder. Rules are data a script checks; the gate verdict is code.

GOOD WHEN

A project ends, or the next team or agent starts.

IN YOUR PROJECT

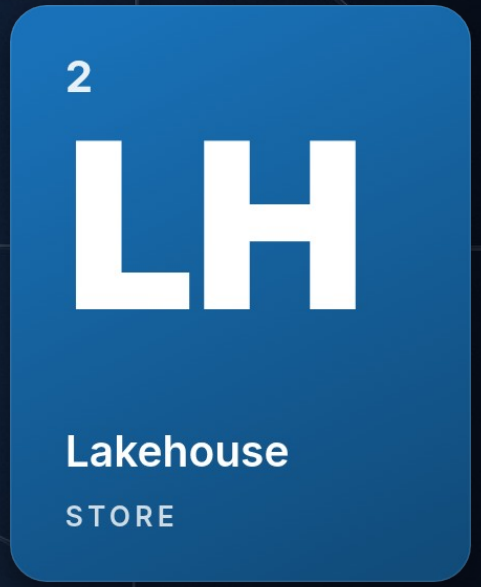
```
wiki/index.md, ADR-*, SEC-*, DRF-*
wiki/rules/ADR-005.yaml
  id: ADR-005
  status: accepted
  decided_by: the human
  check:
    item_names:
      SemanticModel: '^sm_…'
```

TAKEAWAY

A rule belongs in the wiki when a reviewer can check it.

5
Session 2

What changes when the next session starts with the wiki?



STEP 5 · MECHANISM

The next session starts with the wiki

Step 5 - Session 2 plans with the wiki, the review confirms

Plan: Sales request (Giulia, Sales Operations) - session 2

Workspace: FF Ticino Session 2 DEV (a12f95a1-...), empty at start. Identity: ff-demo-contributor (builder).
Request: `task/sales-request.md`. Data: `data/customers.csv` (60 rows, 3 account managers x 20 customers),
`data/orders.csv` (600 rows; all 60 customer ids exist in customers).

Wiki read before planning: ADR-001..ADR-006, the 8 decision pages, SEC-010, SEC-020,
DRF-000, and
`wiki/rules/` (ADR-001, -002, -004, -005, -006 yaml).

Items

Item	Type	Name	Follows
Lakehouse	Lakehouse	`lh_sales` (no schemas)	ADR-005 (type prefixes, a…
Tables	Delta	`customers`, `orders` unde…	ADR-001
Semantic mo…	SemanticMod…	`sm_sales`	ADR-005, ADR-002, ADR-003…
SQL endpoin…	T-SQL objec…	schema `rls`, TVF `rls.fn_…`	ADR-006, SEC-020

Model `sm_sales` (definition in `work/sm_sales/`):

• Direct Lake on OneLake: shared expression `AzureStorage.DataLake("https://onelake.dfs.fabric.microsoft.com/<ws>/<lh>")`,
partitions `mode: directLake`, no `schemaName` (lakehouse without schemas) (ADR-002).
• Star schema (ADR-003): `Customers` (Customer Name, Region, Account Manager, Credit Limit; key hidden),
:

④ Takeaway

The plan in `work/PLAN.md` cites the ADRs and SEC-020 before anything is built. Hand the knowledge on, and keep the review.

FACT CARD · STEP 5

Session 2 starts with knowledge

WHAT

A new session reads the wiki, plans with its ids and builds what earlier findings taught from the start. Reviewers check the result; a human approves.

GOOD WHEN

Every new project, and every new agent on an old one.

IN YOUR PROJECT

```
One line in CLAUDE.md:
  @wiki/index.md, read the
  pages it lists, cite ids
A plan before the build
  cites the ADRs and SEC pages
Reviewers: security,
  rules, drift
A human approves
```

TAKEAWAY

Hand the knowledge on, and keep the review.

Monday morning

Someone renames the model by hand. What does the gate do?

40

DPL

Deployment Pipeline

GOVERN

Step 6 · Monday morning

● Agent "Rules review of deployed state" finished · 58s

› Message from @security-reviewer (ctrl+o to expand)

Ran 1 shell command

● Agent "Security review of deployed state" finished · 1m 5s

● **The gate passed: 0 blocking findings, 0 open questions, 1 info finding.**

The security, rules and drift reviewers all ran as ff-demo-reviewer.

Findings:
- **Info · SEC-010 · baseline · workspace FF Ticino Session 2 DEV**: The reviewer cannot read the workspace role assignments.
  - **Evidence:** GET roleAssignments returned HTTP 403 InsufficientWorkspaceRole.
  - **Remediation:** RLS (row-level security) does not apply to Admin, Member or Contributor, so a human needs to check who holds those roles.

Gate report: reports/20261028-080058-gate.md

I stopped at the gate. The snapshot is signed with ./py gate/snapshot.py sign only after you explicitly say "approve".

✻ Baked for 1m 18s · done 8:01 AM

›

›› auto mode on (shift+tab to cycle) · /tasks to see subagents

④ **Takeaway**

The rule you decided in step 3 also guards the next change. Rename it back, and the review passes.

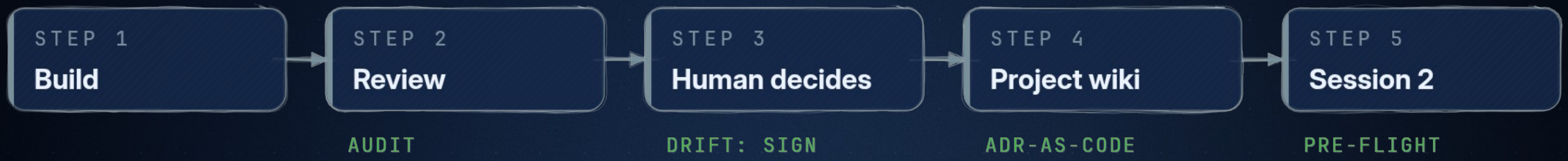
What the gate blocks

BLOCKS	REMEDIATION
SECURITY No RLS role on a table with sensitive data	Add a role that filters its rows, deploy, add members
SECURITY A role without members	Add the members; until then viewers see no data
SECURITY A role member reads other users' rows through another engine	Apply the same rule in every engine
DECISIONS The deployed state contradicts a recorded decision	Change the state back, or decide again and record it
RULES An accepted rule is violated	Follow the rule, or decide a new one and recompile the wiki
DRIFT Changed since the signed baseline	Undo the change, or review it and sign again

The gate stops the sign-off and the promotion, not the build in the DEV workspace.

FOUR PATTERNS

Where the four patterns sit in the steps



BOTH WORKSPACES ARE DEV · THE GATE STANDS BETWEEN DEV AND PRODUCTION

STEP 2

Post-deployment audit

Reviewers read what is deployed and test the effect: it caught a real RLS bypass.

STEP 3 · MONDAY

Drift detection

Signed at step 3, checked against after any change: the twist shows it.

STEP 4

ADR-as-code

A decision in the wiki, written so a reviewer can check it.

STEP 5

Pre-flight review

Session 2 plans against the wiki and builds both rules from the start: passed in round one.

TAKE HOME

For Monday, in three sizes

TALK REPO

1. 30 minutes: CLAUDE.md instruction, Microsoft Learn MCP, a wiki folder
2. A day: reviewers with their own identity, a test identity in the RLS role
3. A sprint: rules, signed baseline, the next session with the wiki

Template in the repo · run it in a sandbox tenant first

SENTENCE OF THE DAY

Agents make good decisions when you let them record, check and hand them on.

You decide what becomes a rule.

Ticino Data Conference 2026

Thank you to our sponsors

Hosted by

acer

Gold

codeway

Lodestar

IT'S TIME TO INNOVATE

VARGROUP

Silver

Bi Factory

DATA KNOWLEDGE ADVISOR

CLOUDMETRIX

EY

K4ANALYTICS®

PLAN ANYWHERE

OKVIZ®

Thank you

Questions now, or later on LinkedIn

Matthias Falland

linkedin.com/in/matthias-falland

LINKEDIN

TALK REPO